信息检索与数据挖掘

实验报告-实验二



山东大学 SHANDONG UNIVERSITY

班　　级	2022 级　人工智能班
学　　号	202200130077
姓　　名	薛润泽
时　　间	2024.11.10

实验内容

实验内容：

本实验旨在实现一个最基本的 Ranked Retrieval Model，即在给定的 tweets 数据集中，根据用户输入的查询（query），返回最相关的前 K 个 tweets。实验要求使用 SMART notation 进行文档和查询的权重计算，并构建倒排索引以加速查询过程。

实验步骤

1、数据准备：

从指定路径读取 tweets 数据，每条 tweet 以 JSON 格式存储。

解析 JSON 数据，提取每条 tweet 的文本内容。

2、构建倒排索引：

初始化变量，包括文档 ID（docid）、倒排索引表（word_to_docid）、所有词集合（allwordsset）、词频文档频率表（DocFc）和词项频率表（TermFc）。

遍历 tweets 数据，对每个 tweet 进行分词处理，并构建倒排索引表和正向索引表。

在构建索引的过程中，同时记录每个词在多少篇文档中出现过（DF）。

3、查询处理：

接收用户输入的查询字符串，将其转换为小写并分词。

构建查询向量 Q，并计算查询向量的权重。权重计算采用对数 TF-IDF 方法，其中 TF 为查询中词项的出现次数，IDF 为词项的逆文档频率。

4、计算余弦相似度：

遍历所有文档，对每个文档构建文档向量，并计算文档向量的归一化值。

计算查询向量与文档向量的余弦相似度，作为文档与查询的相关度得分。

5、排序并输出结果：

根据余弦相似度得分对文档进行排序，选择得分最高的前 K 个文档作为结果输出。

实验代码：

```
import os
import json
import math
import numpy as np

# 设置路径
nd = 'D:/大学/大三上/信息检索与数据挖掘/实验 2'
os.chdir(nd)

# 读取 tweets 数据
with open('tweets.txt', 'r', encoding='utf-8') as file:
    tweets = []
    for line in file:
        try:
            tweet = json.loads(line)
            tweets.append(tweet)
        except json.JSONDecodeError as e:
            print(f"JSONDecodeError: {e}")

# 初始化变量
docid = 1
word_to_docid = {}  # 倒排索引，word -> document
allwordsset = set()
DocFc = {}  # 词 -> 出现的文档数
TermFc = {}  # 词 -> 文档 -> 出现次数

# 构建倒排索引表和正向索引表
for tweet in tweets:
    text = tweet['text']
    spilit_text = text.split()
    tmps = set()
    for s in spilit_text:
        ts = s.lower()
        allwordsset.add(ts)

        # 正向索引表
        if ts not in TermFc:
            TermFc[ts] = {}
        if docid not in TermFc[ts]:
            TermFc[ts][docid] = 1
        else:
            TermFc[ts][docid] += 1
        # 倒排索引表
        if ts not in tmps:
            tmps.add(ts)
            if ts in word_to_docid:
                word_to_docid[ts].append(docid)
            else:
                word_to_docid[ts] = [docid]
```

```
            # 记录 DF
            if ts not in DocFc:
                DocFc[ts] = 1
            else:
                DocFc[ts] += 1

    docid += 1

# 查询输入
query_in = input("Enter query: ")
query_terms = query_in.lower().split()

# 构建查询向量 Q
Q_list = list(set(query_terms))
num_docs = docid - 1
num_terms = len(Q_list)
query_vector = []

# 构建查询向量的权重
for term in Q_list:
    tf = query_terms.count(term)
    idf = math.log10(num_docs / (1 + DocFc.get(term, 0)))
    query_vector.append((1 + math.log10(tf)) * idf if tf > 0 else 0)

# 计算每个文档的向量并求余弦相似度
cosine_scores = {}
for doc_id in range(1, num_docs + 1):
    doc_vector = []
    doc_norm = 0
    for term in Q_list:
        tf = TermFc.get(term, {}).get(doc_id, 0)
        doc_tf_weight = 1 + math.log10(tf) if tf > 0 else 0
        doc_vector.append(doc_tf_weight)
        doc_norm += doc_tf_weight ** 2
    # 归一化文档向量
    doc_norm = math.sqrt(doc_norm)
    doc_vector = [x / doc_norm if doc_norm > 0 else 0 for x in doc_vector]

    # 计算余弦相似度
    cosine_score = np.dot(doc_vector, query_vector)
    cosine_scores[doc_id] = cosine_score

# 排序并输出 Top K 结果
K = 10
top_k_docs = sorted(cosine_scores.items(), key=lambda item: item[1], reverse=True)[:K]
print("Top ",K ,"relevant documents:", top_k_docs)
```

实验总结

本次实验，实现了基于 Ranked Retrieval Model 的信息检索系统。该系统能够接收用户输入的查询字符串，并返回与查询最相关的前 K 个 tweets。在实验过程中，深入认识并实践使用了倒排索引以及 TF-IDF 权重计算和余弦相似度计算。

实验结果表明，该系统能够有效地根据用户查询返回相关文档，并且查询速度较快。

your content